



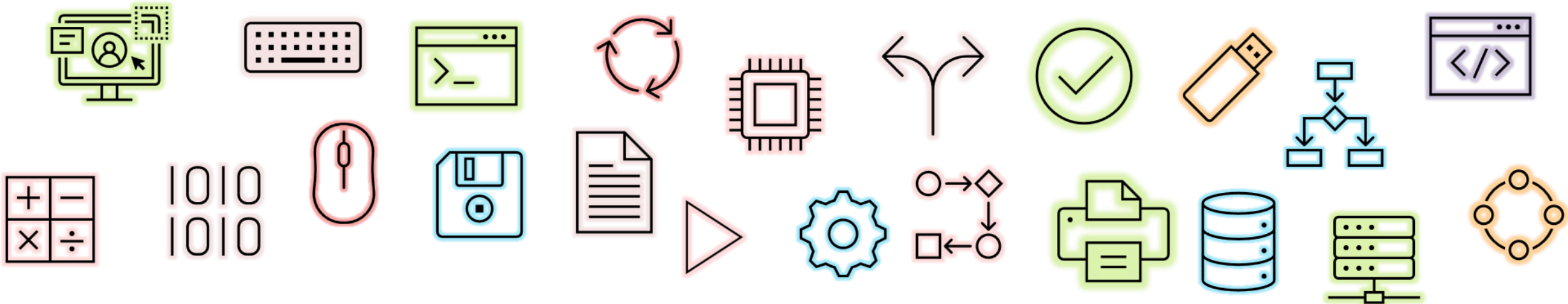
بسم الله الرحمن الرحيم

King Abdulaziz University
Faculty of Computing and information Technology
Computer Science Department



CPCS-202 Programming I

3- Characters and Strings in Java



Objectives

- To represent characters using the **char** type.
- To encode characters using ASCII and Unicode.
- To compare and test characters using the static methods in the **Character** class.
- To represent strings using the **String** objects.
- To return the string length using the **length()** method.
- To use the **+** operator to concatenate strings.
- To read strings from the console.
- To read a character from the console.
- To return a character in the string using the **charAt(i)** method (§4.4.2).
- To obtain substrings.
- To program using characters and strings (**GuessBirthday**).
- To format output using the **System.out.printf** method.

Motivations

- This topic introduces additional basic features of Java these features include Strings and Characters

Character Data Type

- Java allows you to process characters using the character data type, `char`
 - `char` represents a single character
 - a character literal is enclosed in single quotation marks
- Examples:
 - `char letter = 'A';`
 - Assigns the character `A` to the `char` variable `letter`
 - `char numChar = '4';`
 - Assigns the digit character `4` to the `char` variable `numChar`

Unicode and ASCII code

- Computers use binary numbers internally
 - a character is stored in a computer as a sequence of 0s and 1s
 - Mapping a character to its binary representation is called *encoding*
 - There are different ways to encode a character
 - Java uses **Unicode**
 - Unicode is an encoding scheme
 - Originally designed for 16-bit character encoding
 - This allowed for $2^{16} = 65,536$ characters

Unicode and ASCII code

- ASCII
 - Most computers use ASCII
 - American Standard Code for International Exchange
 - 8-bit encoding scheme
 - Used to represent all uppercase and lowercase letters, all digits, all punctuation marks, and control characters
 - 128 characters total
 - Unicode includes ASCII code
 - `\u0000` to `\u007F` representing the 128 ASCII characters

Unicode and ASCII code

- ASCII

TABLE 4.4 ASCII Code for Commonly Used Characters

<i>Characters</i>	<i>Code Value in Decimal</i>	<i>Unicode Value</i>
'0' to '9'	48 to 57	\u0030 to \u0039
'A' to 'Z'	65 to 90	\u0041 to \u005A
'a' to 'z'	97 to 122	\u0061 to \u007A

Check Point

- What is the output of the following program:

```
public class Test {  
    public static void main(String[] args) {  
        char x = 'a';  
        char y = 'c';  
        System.out.println(++x);  
        System.out.println(y++);  
    }  
}
```

O

b
c

The String Type

- A **char** represents one character
- So how do we represent a sequence (a string) of characters?
- Use the data type called **String**
- Example:
 - The following code declares variable **message** to be a **string** with the value "Welcome to Java"
String message = "Welcome to Java";

The String Type

```
String message = "Welcome to Java";
```

- Here, **message** is a reference variable that references a string object with the contents "**Welcome to Java**"

The String Type

- **String** details:
 - Declaring a **String** variable:
`String firstName;`
 - Assign a value to the **String** variable:
`firstName = "Muhammad Ali";`

Simple Methods for **String** Objects

- Getting String Length

- Use the **length()** method to return the number of characters in a string

- Example:

```
String message = "Welcome to Java";  
System.out.println("The length of " +  
    message + " is " + message.length());
```

```
The length of Welcome to Java is 15
```

- Output.

Simple Methods for **String** Objects

- Concatenating Strings

- Remember: you can concatenate a number with a string
- At least one of the operands must be a string
- Examples:

```
String message = "Welcome " + "to " + "Java";
```

```
String s = "Chapter " + 2;
```

– s becomes "Chapter 2"

```
String s1 = "Supplement " + 'B';
```

– s1 becomes "Supplement B"

Simple Methods for **String** Objects

- Concatenating Strings

- The augmented += operator can also be used for concatenation with strings

- Example:

```
String message = "Welcome to Java";  
message += ", and Java is fun.";  
System.out.println(message);
```

```
Welcome to Java, and Java is fun.
```

- C

Simple Methods for **String** Objects

- Concatenating Strings

- Final example:

- If **i** = 1 and **j** = 2, what is the output of the following:
`System.out.println("i + j is " + i + j);`

- Output:

```
i + j is 12
```

- Why?

- In Java, we read from left to right
 - So we have the String "**i + j is** " concatenated with the **int** **i**
 - The result: a new **String** ("**i + j is 1**")
 - This new **String** is concatenated with the **int** **j**
 - You can force addition by enclosing the **i + j** in parentheses

Reading a String from the Console

- How to read a string from the console
 - Use the `next()` method on a Scanner object

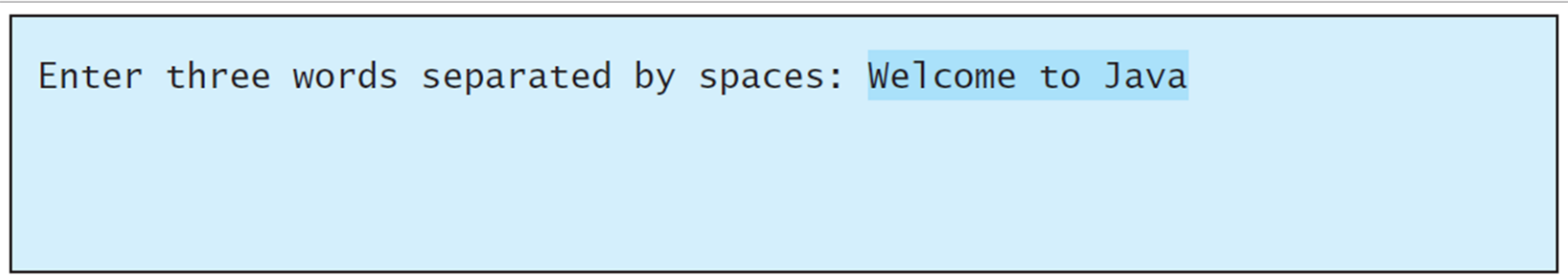
```
System.out.print("Enter three words separated by spaces: ");  
String s1 = input.next(); // assume we made Scanner object  
String s2 = input.next();  
String s3 = input.next();  
System.out.println("s1 is " + s1);  
System.out.println("s2 is " + s2);  
System.out.println("s3 is " + s3);
```

Enter three words separated by spaces:

Reading a String from the Console

- How to read a string from the console
 - Use the `next()` method on a Scanner object

```
System.out.print("Enter three words separated by spaces: ");  
String s1 = input.next(); // assume we made Scanner object  
String s2 = input.next();  
String s3 = input.next();  
System.out.println("s1 is " + s1);  
System.out.println("s2 is " + s2);  
System.out.println("s3 is " + s3);
```

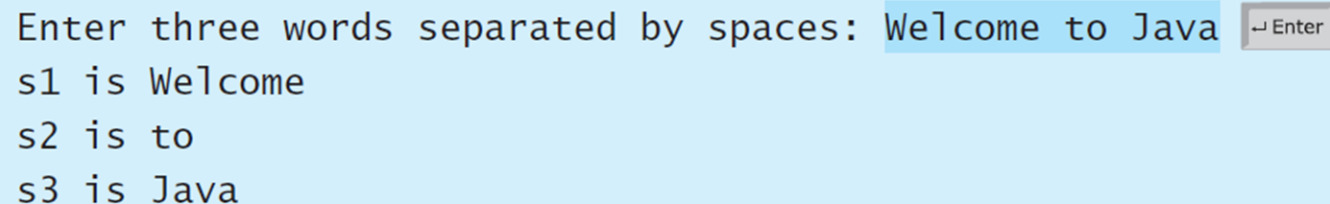


Enter three words separated by spaces: Welcome to Java

Reading a String from the Console

- How to read a string from the console
 - Use the `next()` method on a Scanner object

```
System.out.print("Enter three words separated by spaces: ");  
String s1 = input.next(); // assume we made Scanner object  
String s2 = input.next();  
String s3 = input.next();  
System.out.println("s1 is " + s1);  
System.out.println("s2 is " + s2);  
System.out.println("s3 is " + s3);
```

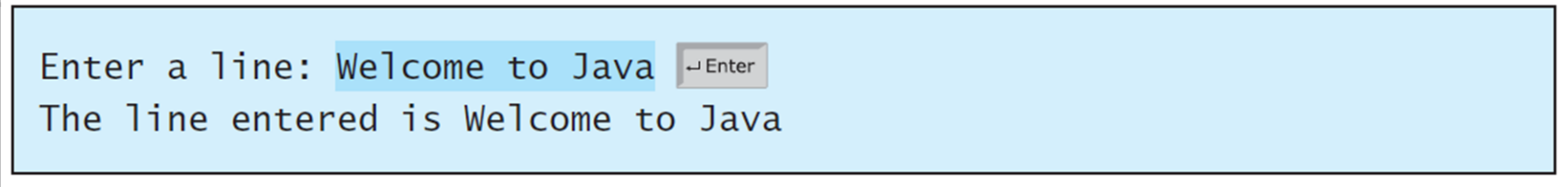


```
Enter three words separated by spaces: Welcome to Java ↵ Enter  
s1 is Welcome  
s2 is to  
s3 is Java
```

Reading a String from the Console

- How to read a complete line from the console
 - Use the `nextLine()` method on a Scanner object

```
Scanner input = new Scanner(System.in);  
System.out.println("Enter a line: ");  
String s = input.nextLine();  
System.out.println("The line entered is " + s);
```



Enter a line: Welcome to Java ↵ Enter
The line entered is Welcome to Java

Check Point

- Show the output of the following expressions:
 - a) `System.out.println("1" + 1);`
 - b) `System.out.println('1' + 1);`
 - c) `System.out.println("1" + 1 + 1);`
 - d) `System.out.println("1" + (1 + 1));`
 - e) `System.out.println('1' + 1 + 1);`

Check Point

- Show the output of the following expressions:
 - a) `System.out.println("1" + 1);` 11
 - b) `System.out.println('1' + 1);` 50
 - c) `System.out.println("1" + 1 + 1);` 111
 - d) `System.out.println("1" + (1 + 1));` 12
 - e) `System.out.println('1' + 1 + 1);` 51

Simple Methods for **String** Objects

- Getting Characters from a String
 - The **s.charAt(index)** method can be used to retrieve a specific character in a string **s**
 - The index is between **0** and **s.length()-1**

Indices	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
message	W	e	l	c	o	m	e		t	o		J	a	v	a

Simple Methods for **String** Objects

- Getting Characters from a String
 - The `s.charAt(index)` method can be used to retrieve a specific character in a string `s`
 - The index is between `0` and `s.length()-1`
 - Example:

`message.charAt(0);` `// Returns the character ...`

Indices	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
message	W	e	l	c	o	m	e		t	o		J	a	v	a

`message.charAt(0)` `message.length()` is ... `message.charAt(...)`

Simple Methods for **String** Objects

- Getting Characters from a String
 - The **s.charAt(index)** method can be used to retrieve a specific character in a string **s**
 - The index is between **0** and **s.length()-1**
 - Example:

message.charAt(0); **// Returns the character W**

Indices	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
message	W	e	l	c	o	m	e		t	o		J	a	v	a

message.charAt(0) message.length() is 15 message.charAt(14)

Reading a Character from the Console

- How to read a single character from the console
 - Use the `nextLine()` method to read a string and then invoke the `charAt(0)` method on the string

```
Scanner input = new Scanner(System.in);
System.out.print("Enter a character: ");
String s = input.nextLine();
char ch = s.charAt(0);
System.out.println("The character entered is " + ch);
```

Common Errors and Pitfalls

- Common Error # 1: Undeclared/Uninitialized Variables and Unused Variables
 - A variable must be declared, and a value assigned to it before you use it.
 - Common errors include not declaring or not initializing a variable
- Example:

```
double increaseRate = 0.05;  
double increase = increaserate * 45;
```

 - Java is case sensitive. In the 2nd line, `increaserate` is not defined, because the `R` is not capitalized

Common Errors and Pitfalls

- Common Error #2: Integer Overflow
 - Remember: numbers are stored with a limited number of digits
 - If you try to save a value that is too large for a variable, an overflow error will occur
- Example:
 - `int value = 2147483647; // allowed`
 - But this number is the biggest possible int, therefore...
 - `value++; // will result in an error`

Common Errors and Pitfalls

- Common Error #3: Round-off Errors
 - a round-off error, also called a rounding error, is the difference between the exact mathematical value of a number and the approximated value that was calculated.
- Example:
 - $1/3$ is approximately 0.333 if you use three decimals and it is 0.3333333 if you use seven decimal places
 - The number of digits you can store is limited
 - So having round-off errors is inevitable (a guarantee!)

Common Errors and Pitfalls

- Common Error #3: Round-off Errors
 - Calculations with floating-point numbers are also approximated
 - Because they are not stored with complete accuracy
 - Example:
 - `System.out.println(1.0 - 0.1 - 0.1 - 0.1 - 0.1 - 0.1);`
 - The output should be 0.5, but the output is really 0.50000000000000001
 - `System.out.println(1.0 - 0.9);`
 - The output should be 0.1, but the output is really 0.099999999999999998
- Message: use integers for exact/precise results!

Common Errors and Pitfalls

- Common Error #4: Unintended Integer Division
 - Perhaps you want “normal” division
 - Example: $3 / 2 = 1.5$
 - However, if both operands are integers, Java will automatically use integer division
 - If you do not want integer division, you should make one of the operands a floating point number

```
int number1 = 1;  
int number2 = 2;  
double average = (number1 + number2) / 2;  
System.out.println(average);
```

(a)

```
int number1 = 1;  
int number2 = 2;  
double average = (number1 + number2) / 2.0;  
System.out.println(average);
```

(b)

Common Errors and Pitfalls

- Common Pitfall #1: Repeated Input Objects
 - New programmers often make many Scanner objects each time they want input
 - This is a mistake!
 - See the code below:

```
Scanner input = new Scanner(System.in);  
System.out.print("Enter an integer: ");  
int v1 = input.nextInt();
```

```
Scanner input1 = new Scanner(System.in);  
System.out.print("Enter a double value: ");  
double v2 = input1.nextDouble();
```

BAD CODE

Common Errors and Pitfalls

- Common Pitfall #1: Repeated Input Objects
 - The code is not wrong, but it is inefficient (slow)
 - It creates two input objects, which is a waste
 - The correct code is below:

```
Scanner input = new Scanner(System.in);  
System.out.print("Enter an integer: ");  
int v1 = input.nextInt();  
System.out.print("Enter a double value: ");  
double v2 = input.nextDouble();
```

GOOD CODE



بسم الله الرحمن الرحيم

King Abdulaziz University
Faculty of Computing and information Technology
Computer Science Department



CPCS-202 Programming I

END

